1. TEAM WORKFLOW, ROLE DIVISION & STORAGE DIRECTORY MAP

VISHAL KHANNA C (68%) & YAASHWANT SKP (32%)

1.1 Project Overview & Collaborative Team Split

This document explains the complete internal working, exact code blocks, file storage locations, and individual engineering contributions for the Plant & Produce Health Analyzer project, developed collaboratively by two teammates:

- 1. Vishal Khanna C — Project Lead, Data Engineering, Deep Learning, Multi-Object CV & Frontend Architect (~68% Contribution): Designed and built the 108-class dataset & priority-sampling pipeline (src/get_data.py, src/preprocess.py), the 2-Stage MobileNetV2 transfer learning architecture & training engine (src/model.py, src/train.py), the HSV/LAB + NMS Multi-Object Localization & Bounding-Box engine (src/detector.py), the complete Streamlit interactive frontend UI (app.py), and the standalone offline launchers (run_app.py, start.bat, start.ps1).
- 2. Yaashwant SKP — Co-Developer: Explainable AI (Grad-CAM), Advisory Rules, OOD Guardrails & Evaluation (~32% Contribution): Designed and built the Grad-CAM convolutional attention heatmap module (src/explainability.py), the 108-class treatment & storage advisory rule engine + Shannon Entropy Out-of-Distribution (OOD) guardrails (src/verdicts.py), the test-split evaluation & confusion matrix script (src/evaluate.py), and the A4 PDF report generator (src/report_generator.py).

File / Folder Path (Where It Is Stored)	Primary Author	Wt %	Core Technical Responsibility
D:\plant-veg-health\data\ (108 folders)	**Vishal Khanna C**	10%	12,000+ raw & augmented leaf/produce images
src/get_data.py & src/preprocess.py	**Vishal Khanna C**	14%	Priority sampling (MAX=150), 70/15/15 .npy splits
src/model.py & src/train.py	**Vishal Khanna C**	16%	MobileNetV2 head + 2-Stage fine-tuning (89.5%–93.3%)
src/detector.py	**Vishal Khanna C**	14%	LAB/HSV segmentation, Canny fallback, NMS & Boxes
app.py, run_app.py, start.bat, start.ps1	**Vishal Khanna C**	14%	2-Tab Streamlit UI, Camera/Batch input, Offline launch
src/explainability.py	**Yaashwant SKP**	9%	Grad-CAM (Conv_1 7x7x1280) GradientTape heatmaps
src/verdicts.py	**Yaashwant SKP**	9%	108-class pathogen/storage rules & OOD Entropy check
src/evaluate.py & model/confusion_ma	**Yaashwant SKP**	7%	Test set evaluation, Precision/Recall/F1 & CM plot
src/report_generator.py	**Yaashwant SKP**	7%	Offline Matplotlib PdfPages A4 diagnostic report

1.3 Where Everything Is Stored on the Computer & GitHub

- Primary Execution Directory: D:\plant-veg-health\ (contains data/, model/, src/, venv/, app.py, start.bat) and reports\
- Workspace Mirror Directory: C:\Users\kurtz\Downloads\MLMProjectFeed\ (synced code, models, and reports)
- Trained Model & Splits Storage: D:\plant-veg-health\model\model.keras (26 MB), model\class_names.txt (108 classes), model\splits\ (X_train.npy, y_train.npy, X_val.npy, y_val.npy, X_test.npy, y_test.npy)
- GitHub Repository: https://github.com/vishalkhannac/crop-health-classifier (Owner: vishalkhannac, Collaborator: yaashwanth95)

2. VISHAL KHANNA C — PART 1: DATASET & PREPROCESSING PIPELINE

FILE LOCATION: D:\plant-veg-health\src\get_data.py & D:\plant-veg-health\src\preprocess.py **Author: Vishal Khanna C** **Lines 1–85**

2.1 How the Dataset Is Organized on Disk (108 Classes)

Vishal unified two major agricultural image sources into D:\plant-veg-health\data\, where each subfolder name is the exact class label: (1) 38 PlantVillage foliar classes (e.g., Tomato___Late_blight, Apple___Apple_scab, Potato___Early_blight, Apple___healthy) and (2) 70 Fresh & Rotten fruit/vegetable classes across 35 produce types (e.g., fresh_apple, rotten_apple, fresh_tomato, rotten_tomato, fresh_carrot, rotten_capsicum).

2.2 Priority-Weighted Class Balancing & Edge-Case Augmentation (src/preprocess.py)

PROBLEM SOLVED BY VISHAL: In raw datasets, some classes had 1,800+ images while others had 120. Moreover, when a user uploaded a sliced rotten apple showing its pale inner core (test_user_apple.jpg), standard models misclassified it as a pear because training data only had uncut whole apples. Vishal generated 80+ sliced apple and multi-apple cluster augmentations and wrote a Priority-Filtered Sampler in src/preprocess.py (Lines 39–52) that caps classes at MAX_PER_CLASS = 150 while guaranteeing that 100% of user_*, sliced_*, and multi_* images are included before random sampling:

```
# src/preprocess.py (Lines 32–54) — Class Discovery & Priority-Weighted Balancing
classes = sorted([d.name for d in DATA_DIR.iterdir() if d.is_dir() and len(get_class_images(d)) > 0])
class_to_idx = {c: i for i, c in enumerate(classes)}

MAX_PER_CLASS = 150  # Balanced representation across all 108 classes
all_paths, all_labels = [], []
for cls in classes:
    imgs = get_class_images(DATA_DIR / cls)
    if len(imgs) > MAX_PER_CLASS:
        # Guarantee retention of sliced cross-sections, multi-object clusters, and user samples
        priority = [p for p in imgs if any(k in p.name.lower() for k in ('user_', 'sliced_', 'multi_'))]
        others   = [p for p in imgs if p not in priority]
        random.seed(SEED)
        sampled_others = random.sample(others, max(0, MAX_PER_CLASS - len(priority)))
        imgs = (priority + sampled_others)[:MAX_PER_CLASS]
    all_paths.extend([str(p) for p in imgs])
    all_labels.extend([class_to_idx[cls]] * len(imgs))
```

2.3 Stratified 70% / 15% / 15% Splitting & NumPy Serialization

Next, in Lines 58–79 of src/preprocess.py, Vishal used scikit-learn's train_test_split with stratify=all_labels twice: first splitting 70% Train and 30% Temp, and then splitting the 30% Temp equally into 15% Validation and 15% Test. The file paths and integer labels are saved into D:\plant-veg-health\model\splits\ as .npy binary arrays so training and evaluation use the exact same deterministic split without data leakage.

```
# src/preprocess.py (Lines 59–79) — Stratified Split & Saving model/labels.txt
X_train, X_temp, y_train, y_temp = train_test_split(
    all_paths, all_labels, test_size=0.30, random_state=SEED, stratify=all_labels)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=SEED, stratify=y_temp)

np.save(splits_dir / 'X_train.npy', X_train); np.save(splits_dir / 'y_train.npy', y_train)
np.save(splits_dir / 'X_val.npy',   X_val);   np.save(splits_dir / 'y_val.npy',   y_val)
np.save(splits_dir / 'X_test.npy',  X_test);  np.save(splits_dir / 'y_test.npy',  y_test)
(MODEL_DIR / 'labels.txt').write_text('\n'.join(classes), encoding='utf-8')
```

HOW VISHAL KHANNA C EXPLAINS THIS TO THE EXAMINER / REVIEW PANEL:

I built the entire data engineering pipeline in src/preprocess.py. To handle class imbalance

3. VISHAL KHANNA C — PART 2: MOBILENETV2 & 2-STAGE TRAINING

FILE LOCATION: D:\plant-veg-health\src\model.py (Lines 1–54) & src\train.py (Lines 1–134) **Author: Vishal Khanna C** **Saved to: model**

3.1 Custom MobileNetV2 Transfer Learning Architecture (src/model.py)

Vishal selected MobileNetV2 (pretrained on ImageNet, 224x224x3 input) because its depthwise separable convolutions and inverted residual linear bottlenecks provide high accuracy with only ~2.25M backbone parameters, allowing fast offline CPU inference. On top of the 7x7x1280 feature map, Vishal added GlobalAveragePooling2D (reducing 7x7x1280 to a 1280-D vector), a Dense(256, ReLU) bottleneck layer, Dropout(0.3) to prevent overfitting, and a final Dense(num_classes, softmax) layer.

```
# src/model.py (Lines 15–36) — Model Architecture Definition
def build_model(num_classes: int, img_size: int = 224) -> tf.keras.Model:
    base = MobileNetV2(input_shape=(img_size, img_size, 3), include_top=False, weights='imagenet')
    base.trainable = False  # Initially frozen for Stage 1 warm-up

    inputs  = tf.keras.Input(shape=(img_size, img_size, 3))
    x       = base(inputs, training=False)          # Output shape: (None, 7, 7, 1280)
    x       = layers.GlobalAveragePooling2D()(x)    # Output shape: (None, 1280)
    x       = layers.Dense(256, activation='relu')(x)
    x       = layers.Dropout(0.3)(x)
    outputs = layers.Dense(num_classes, activation='softmax')(x)
    model   = Model(inputs, outputs, name='plant_veg_health')
    model.base_model = base
    return model
```

3.2 tf.data Pipeline & Two-Stage Fine-Tuning Curriculum (src/train.py)

In src/train.py, Vishal built an asynchronous tf.data.Dataset pipeline (load_image + augment) that resizes images to 224x224, normalizes pixels to [0, 1] by dividing by 255.0, and applies random horizontal flips and +/-10% brightness jitter. To achieve high accuracy without destroying pretrained weights, Vishal implemented a Two-Stage Training Curriculum:

- Stage 1 (Lines 71–85 — Frozen Head Warm-Up, 4 Epochs, lr = 1e-3): Freezes all 154 MobileNetV2 layers and trains only the Dense(256) + Softmax head using Adam(1e-3) and categorical_crossentropy.
- Stage 2 (Lines 87–108 — Top-30 Layer Fine-Tuning, 4 Epochs, lr = 1e-4): Unfreezes the top 30 convolutional layers of MobileNetV2 (model.base_model.layers[:-30] kept frozen) and recompiles with a 10x lower learning rate Adam(1e-4) to specialize high-level filters on disease lesions and fruit decay, reaching 89.51%–93.33% validation accuracy.

```
# src/train.py (Lines 72–111) — Two-Stage Training & Saving model/model.keras
# Stage 1: Warm-up classification head with frozen MobileNetV2 base
model = build_model(num_classes)
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
              loss='categorical_crossentropy', metrics=['accuracy'])
history_1 = model.fit(train_ds, validation_data=val_ds, epochs=4)

# Stage 2: Unfreeze top 30 convolutional layers for domain fine-tuning
model.base_model.trainable = True
for layer in model.base_model.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
              loss='categorical_crossentropy', metrics=['accuracy'])
history_2 = model.fit(train_ds, validation_data=val_ds, epochs=4)
model.save(str(MODEL_DIR / 'model.keras'))
```

HOW VISHAL KHANNA C EXPLAINS THIS TO THE EXAMINER / REVIEW PANEL:

In src/model.py and src/train.py, I implemented a 2-stage transfer learning pipeline with

4. VISHAL KHANNA C — PART 3: MULTI-OBJECT LOCALIZATION ENGINE

FILE LOCATION: D:\plant-veg-health\src\detector.py | Lines 1–222 (Full Module) | Author: Vishal Khanna C

4.1 How Multi-Object Detection Works Without Heavy YOLO Weights

When a user uploads a photo with multiple fruits or vegetables (such as test_multi_apples.jpg containing 3 apples: 2 fresh and 1 rotten), a standard CNN can only output one label for the whole image. Vishal engineered src/detector.py using OpenCV and PIL to automatically detect, isolate, crop, classify, and draw bounding boxes around every object in the photo:

- 1. find_candidate_regions(pil_img) [Lines 7–92]: Converts RGB to HSV and CIE-LAB color spaces. Masks out white/gray backgrounds (L > 215 & S < 45) and dark borders (V < 25). Applies a 7x7 elliptical Morphological Close (2 iterations) and Open (1 iteration). If the background is complex, triggers a Canny Edge Detector fallback (Lines 43–55). Suppresses overlapping boxes using Non-Maximum Suppression (NMS) with overlap_thresh = 0.35 (Lines 60–85).
- 2. extract_crop_with_padding(pil_img, box, padding_ratio=0.08) [Lines 95–107]: Adds an 8% spatial margin around each bounding box so the edges of the fruit or leaf are preserved when resized to 224x224.
- 3. draw_bounding_boxes(pil_img, item_results) [Lines 110–152]: Draws thick Green rectangles RGB(46, 204, 113) for Safe items and Red rectangles RGB(231, 76, 60) for Rotten/Diseased items, with a top label badge showing '#1: Fresh Apple (85.1%)'.
- 4. analyze_image_multiobject(...) [Lines 155–222]: Loops through all detected boxes, runs model.predict() on each 224x224 crop, collects per-item verdicts, and computes the overall Batch Summary Verdict.

```
# src/detector.py (Lines 13–30 & 69–85) — Color Segmentation & Non-Maximum Suppression (NMS)
img_hsv = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV)
img_lab = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2LAB)
is_white_bg = (img_lab[:, :, 0] > 215) & (img_hsv[:, :, 1] < 45)
is_black_bg = (img_hsv[:, :, 2] < 25)
fg_mask = ~(is_white_bg | is_black_bg)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
mask_clean = cv2.morphologyEx(fg_mask.astype(np.uint8)*255, cv2.MORPH_CLOSE, kernel, iterations=2)

# Non-Maximum Suppression (NMS) at IoU / Overlap Threshold = 0.35
overlap_thresh = 0.35
while len(idxs) > 0:
    i = idxs[0]; pick.append(i)
    xx1 = np.maximum(x1[i], x1[idxs[1:]]); yy1 = np.maximum(y1[i], y1[idxs[1:]])
    xx2 = np.minimum(x2[i], x2[idxs[1:]]); yy2 = np.minimum(y2[i], y2[idxs[1:]])
    overlap = (np.maximum(0, xx2 - xx1) * np.maximum(0, yy2 - yy1)) / area[idxs[1:]]
    idxs = np.delete(idxs, np.concatenate(([0], np.where(overlap > overlap_thresh)[0] + 1)))
```

```
# src/detector.py (Lines 161–203) — Per-Object Crop Inference & Batch Verdict Synthesis
for i, box in enumerate(boxes, start=1):
    crop = extract_crop_with_padding(pil_img, box).resize((224, 224))
    arr  = np.expand_dims(np.array(crop, dtype=np.float32) / 255.0, axis=0)
    preds = model.predict(arr, verbose=0)[0]
    top_idx = int(np.argmax(preds)); conf = float(preds[top_idx]) * 100.0
    v_info = get_verdict_func(labels[top_idx], confidence=conf, preds=preds)

if rotten_count == 0:   batch_verdict = 'All items appear fresh and safe to consume.'
elif fresh_count == 0:  batch_verdict = 'All detected items exhibit signs of rot/disease — do not consume.'
else: batch_verdict = f'Mixed Batch: {fresh_count} fresh, {rotten_count} rotten. Discard affected pieces.'
```

HOW VISHAL KHANNA C EXPLAINS THIS TO THE EXAMINER / REVIEW PANEL:

To support uploading multiple fruits or vegetables in a single frame without requiring a heavy YOLO checkpoint, I wrote src/detector.py. It segments foreground objects in CIE-LAB

FILE LOCATION: D:\plant-veg-health\app.py (Lines 1–374) & run_app.py / start.bat | Author: Vishal Khanna C | Port: localhost:85

5.1 Streamlit Web Application Architecture (app.py)

Vishal built the entire user-facing web application in app.py using Streamlit and custom CSS card styling. To guarantee zero-latency interaction, load_model_and_labels() (Lines 92–114) is decorated with @st.cache_resource so model.keras is loaded into RAM only once at startup.

- Custom CSS & Card System (Lines 43–89): Defines color-coded visual cards: .card-safe (green #e8f5e9), .card-unsafe (red #ffebee), .card-mixed (amber #fff8e1), .card-class (blue #e3f2fd), and .card-disease (magenta #fce4ec).
- Tab 1 — Live Diagnosis & Batch Inspector (Lines 142–342): Provides 3 interactive input modes via st.radio: (1) Upload Photo(s) with accept_multiple_files=True and an automated Multi-File Batch Summary Dataframe; (2) Live Camera Capture via st.camera_input(); and (3) Quick Demo Samples with 7 pre-loaded benchmark images. Displays side-by-side Annotated Bounding Boxes + Grad-CAM Heatmaps, expandable per-item breakdowns, and the PDF Download button.
- Tab 2 — Model Architecture & Evaluation Metrics (Lines 344–371): Displays 4 KPI metric cards, the Training & Validation Curves image, the Confusion Matrix image, and a 3-column searchable directory of all 108 classes.

```
# app.py (Lines 91–114 & 204–234) — Cached Model Loader & Main Inference Loop
@st.cache_resource(show_spinner='Loading deep learning model & class index …')
def load_model_and_labels():
    model  = tf.keras.models.load_model(str(model_path))
    labels = [l.strip() for l in labels_path.read_text(encoding='utf-8').splitlines() if l.strip()]
    return model, labels, model_path.parent

# Inside Tab 1 (col_left): Run Multi-Object Detector + Grad-CAM + PDF Export
cur_name, pil_img = active_images[selected_idx]
analysis    = detector.analyze_image_multiobject(pil_img, model, labels, get_verdict)
gradcam_img = explainability.generate_gradcam(pil_img, model) if show_gradcam else None
pdf_bytes   = report_generator.generate_pdf_report(pil_img, analysis, gradcam_img)
st.download_button('□ Download Official PDF Diagnostic Report', data=pdf_bytes, mime='application/pdf')
```

5.2 100% Standalone Offline Launchers (run_app.py, start.bat, start.ps1)

To ensure the project runs completely offline on any Windows machine without terminal setup errors, Vishal created run_app.py and start.bat. They disable external telemetry (STREAMLIT_BROWSER_GATHER_USAGE_STATS=false, HF_HUB_OFFLINE=1), automatically locate the embedded Python executable and venv/Lib/site-packages across C: and D: drives, and launch Streamlit on port 8501.

```
# run_app.py (Lines 1–31) — Dynamic Path Resolution & Programmatic Streamlit CLI
candidate_site_packages = [ROOT / 'venv' / 'Lib' / 'site-packages', Path(r'D:\plant-veg-health\venv\Lib\site-packages')]
for p in candidate_site_packages:
    if p.exists() and str(p) not in sys.path: sys.path.insert(0, str(p))
import streamlit.web.cli as stcli
if __name__ == '__main__':
    sys.argv = ['streamlit', 'run', str(ROOT / 'app.py'), '--server.port=8501', '--server.headless=true']
    sys.exit(stcli.main())
```


HOW VISHAL KHANNA C EXPLAINS THIS TO THE EXAMINER / REVIEW PANEL:

I developed the Streamlit web interface in app.py and the offline launchers in run_app.py and start.bat. I used @st.cache_resource for instant model loading, built three input modes (Multi-file upload, Live Webcam, and Quick Demo Samples), integrated Yaashwant's Grad-CAM and advisory modules into the UI cards, and configured start.bat for 1-click 100% offline execution.

6. YAASHWANT SKP — PART 1: GRAD-CAM VISUAL EXPLAINABILITY ENGINE

FILE LOCATION: D:\plant-veg-health\src\explainability.py | Lines 1–68 (Full Module) | Author: Yaashwant SKP

6.1 Why Grad-CAM Explainability Is Essential

Deep neural networks are often criticized as 'black boxes' because they output a class probability without showing which part of the leaf or fruit triggered the prediction. Yaashwant SKP implemented src/explainability.py using Gradient-weighted Class Activation Mapping (Grad-CAM) to visually highlight the exact fungal lesions, necrotic spots, or rotting tissue regions that activated the MobileNetV2 model.

6.2 Step-by-Step Code Walkthrough of generate_gradcam() (src/explainability.py)

- Step 1 — Backbone & Head Layer Separation (Lines 20–32): Iterates through model.layers to separate the nested MobileNetV2 functional backbone (which outputs the 7x7x1280 spatial feature map from Conv_1 / out_relu) from the top classification head layers (GlobalAveragePooling2D -> Dense(256) -> Dropout -> Dense(108)).
- Step 2 — GradientTape Forward Pass & Watch (Lines 34–42): Passes the normalized 1x224x224x3 image tensor through base_model inside a tf.GradientTape() context, explicitly calling tape.watch(conv_outputs), and forwards conv_outputs through the head layers to obtain the top predicted class score preds[:, top_class_idx].
- Step 3 — Spatial Gradient Pooling & ReLU Weighting (Lines 44–56): Computes grads = tape.gradient(top_class_Channel, conv_outputs) of shape (1, 7, 7, 1280). Global-average-pools the gradients over height and width axes (0, 1, 2) to get a 1280-D channel importance vector pooled_grads, multiplies each of the 1280 feature map channels by its importance weight, sums along axis=-1, and applies tf.maximum(heatmap, 0) (ReLU) so only positive class evidence is kept.
- Step 4 — Bicubic Upsampling & JET Colormap Blending (Lines 58–66): Resizes the 7x7 heatmap to the original image dimensions (orig_w, orig_h) using cv2.INTER_CUBIC, applies cv2.COLORMAP_JET (Red/Yellow = high AI attention, Blue = background), and alpha-blends it with the original RGB image.

```
# src/explainability.py (Lines 20–66) — Complete Grad-CAM Implementation by Yaashwant SKP
# Separate MobileNetV2 convolutional base from classification head layers
base_model, head_layers, found_base = None, [], False
for layer in model.layers:
    if not found_base and isinstance(layer, tf.keras.Model):
        base_model = layer; found_base = True
    elif found_base:
        head_layers.append(layer)

with tf.GradientTape() as tape:
    conv_outputs = base_model(img_tensor, training=False)  # Shape: (1, 7, 7, 1280)
    tape.watch(conv_outputs)
    x = conv_outputs
    for hl in head_layers: x = hl(x, training=False)
    top_class_idx = tf.argmax(x[0])
    top_class_Channel = x[:, top_class_idx]

grads = tape.gradient(top_class_Channel, conv_outputs)
pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))       # Channel weights: (1280,)
heatmap = tf.reduce_sum(tf.multiply(conv_outputs[0], pooled_grads), axis=-1)
heatmap = tf.maximum(heatmap, 0) / (tf.reduce_max(heatmap) + 1e-12)

heatmap_resized = cv2.resize(heatmap.numpy(), (orig_w, orig_h), interpolation=cv2.INTER_CUBIC)
colormap_rgb = cv2.cvtColor(cv2.applyColorMap(np.uint8(255*heatmap_resized), cv2.COLORMAP_JET), cv2.COLOR_BGR2RGB)
blended = (orig_rgb * (1.0 - weight_map) + colormap_rgb * weight_map).astype(np.uint8)
```

HOW YAASHWANT SKP EXPLAINS THIS TO THE EXAMINER / REVIEW PANEL:

7. YAASHWANT SKP — PART 2: ADVISORY ENGINE & OOD GUARDRAILS

FILE LOCATION: D:\plant-veg-health\src\verdicts.py | Lines 1–368 (Full Module) Author: Yaashwant SKP

7.1 Deterministic 108-Class Verdict, Treatment & Storage Engine (src/verdicts.py)

Because neural networks only output a class index, Yaashwant SKP engineered the domain knowledge engine in src/verdicts.py that translates any of the 108 class names into an 8-key dictionary: {status, disease, safe, verdict, note, treatment, prevention, is_ood}.

- 1. _PLANT_VERDICTS Dictionary (Lines 13–231): Maps all 38 PlantVillage classes to their scientific pathogen binomial (e.g., Phytophthora infestans, Venturia inaequalis, Xanthomonas campestris), safety verdict, organic/chemical treatment (copper fungicide, neem oil, sulfur, pruning), and crop-rotation prevention.
- 2. _STORAGE_GUIDE & _PRODUCE_VERDICTS (Lines 236–305): Maps all 35 produce types (70 fresh/rotten classes) to exact temperature (°C), humidity, chilling-injury warnings (for bananas, mangoes, tomatoes, potatoes), and ethylene gas separation rules (e.g., keeping apples away from carrots).

7.2 Out-of-Distribution (OOD) & Shannon Entropy Guardrail (Lines 318–368)

PROBLEM SOLVED BY YAASHWANT: If someone uploads a non-plant photo (e.g., a chair or a blurry image), a standard softmax classifier still picks a class. Yaashwant wrote check_ood_guardrail(preds) using both the top-1 probability and Shannon Entropy H(p) = -Sum(p_i * log(p_i)) to detect flat/uncertain probability distributions and flag them as Unrecognized Objects:

```
# src/verdicts.py (Lines 318–365) — Shannon Entropy OOD Guardrail & Verdict Lookup
def check_ood_guardrail(preds: np.ndarray, conf_threshold: float = 28.0, entropy_threshold: float = 4.15) -> dict:
    top_conf = float(np.max(preds)) * 100.0
    probs    = np.clip(preds, 1e-12, 1.0)
    entropy  = float(-np.sum(probs * np.log(probs)))  # Shannon Entropy across 108 classes
    is_ood   = (top_conf < conf_threshold) or (entropy > entropy_threshold and top_conf < 45.0)
    return {'is_ood': is_ood, 'top_confidence': top_conf, 'entropy': entropy}

def get_verdict(class_name: str, confidence: float = None, preds: np.ndarray = None) -> dict:
    if preds is not None and check_ood_guardrail(preds)['is_ood']:
        return dict(status='Unrecognized / Low Confidence Object', safe=False,
                    verdict='Uncertain — Retake Photo', is_ood=True, ...)
    status, disease, safe, verdict, note, treatment, prevention = _ALL_VERDICTS.get(class_name, _DEFAULT)
    return dict(status=status, disease=disease, safe=safe, verdict=verdict,
                note=note, treatment=treatment, prevention=prevention, is_ood=False)
```

HOW YAASHWANT SKP EXPLAINS THIS TO THE EXAMINER / REVIEW PANEL:

I built src/verdicts.py to convert raw model predictions into actionable agricultural and food-safety intelligence. I mapped all 38 foliar classes to pathogen-specific treatments and all 70 produce classes to temperature and ethylene storage rules. I also implemented check_ood_guardrail() using Shannon Entropy H(p) = -Sum(p log p) so that blurry or non-produce images are safely intercepted instead of producing false diagnoses.

8. YAASHWANT SKP — PART 3: EVALUATION, PDF GENERATOR & VIVA GUIDE (CONTRIBUTED BY YAASHWANT SKP)

FILE LOCATION: D:\plant-veg-health\src\evaluate.py (Lines 1–73) & src\report_generator.py (Lines 1–133) | **Author: Yaashwant SKP** | **Ou**

8.1 Model Evaluation & Confusion Matrix Generator (src/evaluate.py)

In src/evaluate.py, Yaashwant loaded the held-out 15% test split (model/splits/X_test.npy, y_test.npy), built a batched tf.data pipeline, ran model.evaluate() and model.predict(), generated scikit-learn's classification_report (per-class Precision, Recall, and F1-score), and plotted the multi-class ConfusionMatrixDisplay saved to model/confusion_matrix.png.

```
# src/evaluate.py (Lines 52–72) — Test Evaluation & Confusion Matrix Plot
loss, acc = model.evaluate(test_ds, verbose=1)
y_pred = np.argmax(model.predict(test_ds, verbose=0), axis=1)
print(classification_report(eval_y, y_pred, target_names=labels, zero_division=0))
cm = confusion_matrix(eval_y, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot(ax=ax, xticks_rotation='vertical', colorbar=False)
plt.savefig(str(MODEL_DIR / 'confusion_matrix.png'), dpi=80)
```

8.2 Standalone Offline A4 PDF Report Generator (src/report_generator.py)

In src/report_generator.py, Yaashwant used Matplotlib's PdfPages and io.BytesIO to dynamically draw an A4 portrait PDF (figsize=(8.27, 11.69)) containing the green header banner, side-by-side Annotated Specimen and Grad-CAM images, confidence & verdict rows, treatment/storage advisories, and the mandatory food-safety disclaimer, returning raw PDF bytes to Streamlit's st.download_button.

```
# src/report_generator.py (Lines 14–53 & 127–133) — In-Memory A4 PDF Report Generation
def generate_pdf_report(pil_img: Image.Image, analysis: dict, gradcam_img: Image.Image = None) -> bytes:
    buf = io.BytesIO()
    with PdfPages(buf) as pdf:
        fig = plt.figure(figsize=(8.27, 11.69), dpi=150)  # A4 Portrait
        ax_img1.imshow(analysis['annotated_image']); ax_img2.imshow(gradcam_img)
        # Renders Identified Class, Confidence %, Disease, Verdict, Treatment & Storage
        pdf.savefig(fig); plt.close(fig)
    buf.seek(0); return buf.getvalue()
```

8.3 Quick Presentation & Viva Hand-Off Script for Tomorrow

- Step 1 (Vishal starts — 3 mins): Introduces the problem, explains the 108-class dataset and Priority-Weighted Sampler in src/preprocess.py, explains the 2-Stage MobileNetV2 architecture in src/model.py & src/train.py, demonstrates the Multi-Object HSV/LAB + NMS bounding-box detector in src/detector.py using the 3-apple demo in app.py, and shows the offline launcher start.bat.
- Step 2 (Yaashwant follows — 2 mins): Toggles on the Grad-CAM Heatmap in the UI and explains src/explainability.py (GradientTape on Conv_1 7x7x1280), opens the Treatment & Storage Advisory card and explains src/verdicts.py + the Shannon Entropy OOD guardrail, shows the Confusion Matrix in Tab 2 (src/evaluate.py), and clicks 'Download Official PDF Diagnostic Report' (src/report_generator.py).